

BPC-PRP Robot development

1st Jan Drochýtek

bachelor student of Automation and Measurement
FEEC Brno University of Technology
 Brno, Czechia
 jan.drochytsek@vut.cz

2nd Marek Holečka

bachelor student of Automation and Measurement
FEEC Brno University of Technology
Brno, Czechia
marek.holecka@vut.cz

Abstract—This work presents the development of a mobile robot capable of autonomously escaping a maze by combining corridor navigation with ArUco-based localization. After introducing the development environment and hardware platform, the individual subsystems of the robot are incrementally designed and implemented, including motor control, sensor acquisition, camera integration, and communication interfaces. On this foundation, navigation and control algorithms are developed, starting from basic motion primitives and progressing to reliable line and corridor following in constrained environments. Finally, ArUco marker detection is integrated to support robust pose estimation and decision making at intersections, enabling the robot to construct a high-level representation of the maze and select appropriate escape routes. The resulting system demonstrates how modular component development can be combined into a cohesive solution for autonomous maze solving.

Index Terms—Autonomous Navigation, Sensor Fusion, ROS2, Differential Drive, Exponential Moving Average

I. INTRODUCTION

The focus of this project is the development of an autonomous mobile robot capable of reliable navigation in a structured corridor-based maze environment. The robot is required to follow corridors, handle intersections, and successfully escape the maze using ArUco marker-based decision making, with a fallback behavior based on predefined logic.

The system is implemented within the ROS2 framework, which enables modular communication between sensors, control algorithms, and actuators. Sensor data is fused to provide robust perception of the environment and ensure stable navigation even under noisy conditions.

The robot uses a state-based navigation architecture, where low-level control (PID regulation of heading and motor control) is combined with high-level decision making, such as intersection handling and ArUco-based direction selection. The implemented solution focuses on robustness, filtering of sensor noise, and fault tolerance in case of partial sensor failure.

II. SYSTEM ARCHITECTURE

The robot is based on the Fenrir platform. It uses a Raspberry Pi 4 as a main computer supplemented with an Arduino Nano as a microcontroller. Two 10,000 mAh Alza powerbanks were used as a power source. These powerbanks are powering two motors with encoders (DG01D-E) through a voltage regulator. Both motors form an inseparable part of the differential drive. For navigation inside a corridor, a 2D

LIDAR (RPLiDAR A1M8) was used. Turning and centering is done based on data from IMU - MPU6050. The project is based on the ROS2 framework which handles communication between nodes and data transfer between the microcontroller, sensors, and actuators.[2]

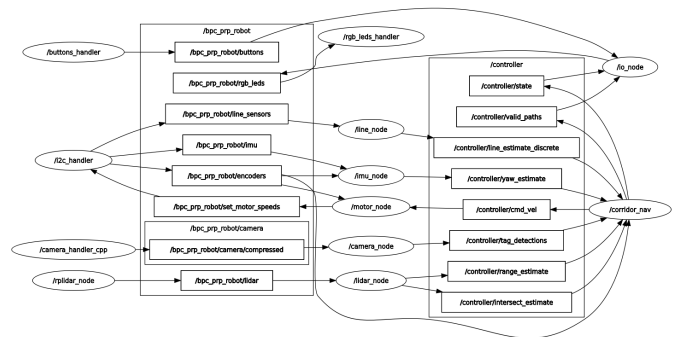


Fig. 1. Communication structure between nodes

III. DATA PROCESSING AND SENSOR FUSION

A. Heading Calculation

To ensure an accurate and drift-free heading (ψ), the system utilizes data fusion.

- **Complementary Filter:** The integrated yaw from the IMU is fused with the yaw calculated from the wheel encoders. This approach allows us to utilize the quick reaction time gyroscope while reducing the integration error from IMU.

B. Infrared Line Sensors

- **EMA Filtering:** Raw values from the line sensors are smoothed using an Exponential Moving Average (EMA).
- **Memory Efficiency:** Unlike standard moving averages, the EMA does not require maintaining a buffer of historical values, which allows for a simpler implementation. EMA is calculated using the following formulas: [1]

$$EM A_t = \alpha \cdot P_t + (1 - \alpha) \cdot EM A_{t-1} \quad (1)$$

$$\alpha = \frac{2}{N+1} \quad (2)$$

- EMA_t - current value of the Exponential Moving Average (EMA) at time t

- P_t - current value at time t
- EMA_{t-1} - previous EMA value
- α - smoothing factor determining the weight of the current value
- N - number of periods used for the EMA calculation

The used value for α is 0.90.

C. LIDAR Filtering

- **Arithmetic Mean:** LIDAR data is filtered using an arithmetic mean to reduce environmental noise.
- **Dual Averaging:** The system maintains two separate averages: one evaluated for the general surroundings and intersection detection with an angle width of 25° and another strictly for detecting free space at the intersection with an angle width of 15° . Decreasing the lobe width helped eliminate early intersection detection.

IV. MOTION CONTROL AND NAVIGATION

A. PID Control and Yaw Regulation

The core corridor navigation relies on maintaining a constant heading. Two independent PID regulators were used. PID for centering between walls has components with the following values in order K_p , K_i , K_d , τ .

- PID_{yaw} component values: 4, 0, 0, $3 \cdot 10^{-3}$
- PID_{turn} component values: 6, 0.5, 3, $3 \cdot 10^{-3}$

Filtered Derivative Component: To prevent jerky motor behavior caused by high-frequency sensor noise, the derivative (D) component of the PID controller is filtered. The filtering coefficient is τ , the value is typically 3 to 5 times Δt .

B. LIDAR Biasing

- **Error Adjustment:** The baseline yaw error is dynamically adjusted based on LIDAR wall proximity.
- **Wall Avoidance:** If the robot exceeds a threshold of 20 cm from a wall, the error term is modified to gently steer the robot back to the center line, preventing contact with the wall and ensuring correct turning at an intersection or at a turn. Furthermore, keeping the robot centered helps with better scanning of ArUco tags. To keep the robot from ramming to a wall a threshold of 28 cm was chosen.

C. Motor Control

- **Ramp Profiles:** To prevent wheel slip and reduce mechanical stress, all motor velocity commands are smoothed using Ramp-Up and Ramp-Down functions which limit the maximum acceleration. The used limits are 0.8 ms^{-2} for linear acceleration and 12 rads^{-1} for angular acceleration.

V. INTERSECTION LOGIC AND SAFETY

A. Intersection Entry and Correction

- **Encoder Measurement:** The robot navigates into the center of an intersection based on the traveled distance measured by the encoders, which is 22 cm, the value was selected to be 15 cm distance travelled plus the distance

from the line sensors to the motors which is 7 cm. For exiting the intersection a distance of 0.35 cm is chosen.

- **Line Correction:** To correct too early intersection detections, the line detectors are used. When a perpendicular line is detected, the measured encoder distance is reset to zero. This approach makes sure the robot always starts the turn in the center of the intersection.

B. Failsafe Mechanisms

- **LIDAR Watchdog:** If the time since the last LIDAR message exceeds 175ms, the system triggers a safety stop and waits for data restoration before continuing its movement.

VI. SYSTEM ARCHITECTURE AND FEEDBACK

A. ROS2 Services and Logging

- **Service Calls:** System commands are handled via ROS2 services. This includes IMU management (Calibrate, Reset) and IO interrupts, such as changing the state machine via a physical button. Button 1 puts the robot into the CALIBRATION state, followed by CORRIDOR_FOLLOWING. Button 2 completely stops the robot. These buttons helped us save a substantial amount of time during the debugging process.
- **State Logging:** The entire state machine is actively logged to the terminal informing us of the current state of the robot.

B. Visual Diagnostics (LEDs)

The robot utilizes four distinct LEDs for real-time debugging:

- **State Mode:**

TABLE I
MAPPING OF CORRIDOR STATES TO COLORS

State	Color
WAIT	WHITE
CALIBRATION	BLUE
CORRIDOR_FOLLOWING	YELLOW
INTERSECTION	GREEN
INTERSECTION_ADVANCE	RED
EXIT_INTERSECTION	CYAN
TURNING	MAGENTA
RESET	ORANGE

- **Intersection Mode:** Upon detecting a junction, the LEDs switch to indicate which specific directions (left, right, straight) are open.

VII. REMOTE CONTROL

The Joy node, a remote control interface using a gamepad, was implemented as a voluntary exercise during the development. The gamepad was used for empirical validation of correctly implemented kinematics and motor control.

TABLE II
LED SIGNALING BEHAVIOR IN DIFFERENT ROBOT STATES

State	LED Index	Meaning
PATH / Intersection	LED[1]	Right path available (WHITE)
	LED[2]	Straight path available (WHITE)
	LED[3]	Left path available (WHITE)
	LED[0]	Not used in this mode
LINE / Navigation	LED[1], LED[2]	No deviation (OFF)
	LED[2]	Slight left deviation (ORANGE)
	LED[1]	Slight right deviation (ORANGE)
	LED[1], LED[2]	Strong deviation (ORANGE both)
	LED[0]	Current system state indicator

VIII. CONCLUSION

The robot successfully navigated the maze within a five-minute timeframe. Initial difficulties with corridor navigation relying solely on LIDAR were resolved by integrating IMU data for robust sensor fusion. Sensor noise was effectively mitigated using a complementary filter for the heading. Furthermore, two separate PID controllers were implemented to manage turning and yaw regulation independently. By decreasing the LIDAR lobe width, intersection detection became significantly more robust, eliminating false positives. The system also successfully leveraged ArUco markers for high-level decision-making at junctions. Future iterations of this project could focus on optimizing the control logic to reduce overall traversal time, while the implementation of Simultaneous Localization and Mapping (SLAM) techniques would enable fully autonomous navigation in unmapped environments.

REFERENCES

- [1] NIST/SEMATECH. *6.4.3.1. Single Exponential Smoothing*. e-Handbook of Statistical Methods. Accessed: 2026-05-21. 2012. URL: <https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc431.htm>.
- [2] Robotics-BUT. *Fenrir Project Repository*. GitHub repository, accessed: 2026-05-21. 2026. URL: <https://github.com/Robotics-BUT/fenrir-project>.